Zomato / Eternal

OA & Interview Solutions

Part I — Actual Coding Questions in C++

Part II — 55 Advanced ML MCQs with Solutions

Campus Placement Preparation — 2026 Cycle

Fresh Graduate / SDE-1 / ML Engineer

Compiled from publicly shared OA and interview experiences (2021–2026).
Solutions, derivations and MCQs written from scratch.

Contents

1 How the Assessment Is Structured

Platform: HackerRank **Duration:** 90 minutes
Sections: 13–15 CS-Fundamentals MCQs (5 marks each, no negative marking) + 3 coding questions (100 marks each).
MCQ spread: sorting internals (∼3), graphs (∼2), DBMS (∼3–4, some multiple-correct), problem solving (∼2), OS/misc.
Coding spread: one greedy/heap, one DP or tree, one array/sweep-line. Difficulty is solid Medium, occasionally Hard.

Time strategy. MCQs are worth ∼65–75 marks total and take 15 minutes. Coding is worth 300. Do MCQs first at speed, then spend 25 minutes per coding question. Partial test cases still score, so always submit a brute force before optimising.

Part I — Coding Questions

2 Interval Overlap Count

Problem

Given arrays `start[]` and `end[]` where `(start[i], end[i])` defines an interval, report the overlap structure. Constraints: $n \le 2 \times 10^5$, coordinates up to 10^9.

Two readings appear in different recollections of this question, so both are solved below.

Variant A — Maximum number of intervals overlapping at any point

Key idea. An interval only changes the "how many are active" counter at two places: it $+1$ at `start` and -1 just after `end`. Sort all $2n$ events, sweep left to right, and track the running counter. Coordinates up to 10^9 never need compression if you sort events rather than building an array.

Pattern: Sweep line / difference array **Time:** $O(n \log n)$ **Space:** $O(n)$

```cpp
#include <bits/stdc++.h>
using namespace std;

int maxOverlap(vector<int>& start, vector<int>& end) {
    int n = start.size();
    vector<pair<long long,int>> ev; // (coordinate, delta)
    ev.reserve(2 * n);
    for (int i = 0; i < n; ++i) {
        ev.push_back({start[i], +1});
        ev.push_back({(long long)end[i] + 1, -1}); // closed interval
    }
    // At equal coordinate, process -1 before +1? No: a closed interval
    // [1,3] and [3,5] DO overlap at x = 3, so we shifted the end by +1
    // and can safely sort by (coord, delta) with -1 first.
    sort(ev.begin(), ev.end());

    int cur = 0, best = 0;
    for (auto& [x, d] : ev) { cur += d; best = max(best, cur); }
    return best;
}
```

Variant B — Number of overlapping *pairs*

Key idea. Sort intervals by start. Sweep with a min-heap of active end points. Before adding interval i, pop every end point $< start_i$; whatever remains in the heap overlaps with i, so add `heap.size()` to the answer. Each interval enters and leaves the heap once.

Pattern: Sort + min-heap **Time:** $O(n \log n)$ **Space:** $O(n)$

```cpp
long long overlappingPairs(vector<int>& start, vector<int>& end) {
    int n = start.size();
    vector<pair<long long,long long>> iv(n);
    for (int i = 0; i < n; ++i) iv[i] = {start[i], end[i]};
    sort(iv.begin(), iv.end());

    priority_queue<long long, vector<long long>, greater<long long>> active;
    long long ans = 0;
    for (auto& [s, e] : iv) {
```

```
        while (!active.empty() && active.top() < s) active.pop();
        ans += (long long)active.size(); // all still-active overlap with [s,e]
        active.push(e);
    }
    return ans;
}
```

Trap. `ans` can reach $\binom{2\times10^5}{2} \approx 2 \times 10^{10}$. Using `int` is the single most common wrong answer here.

3 Bank Transaction — Maximum Count with Non-Negative Running Balance

Problem

Given `t[0..n-1]` (values in $[-10^9, 10^9]$), select the *maximum number* of transactions, keeping their original order, such that the running total never drops below zero at any prefix of the selection.

Key idea — regret greedy. Scan left to right and *take everything*. Push each taken value into a min-heap and add it to the balance. The moment the balance goes negative, undo the single worst decision made so far: pop the minimum (most negative) element and add its magnitude back.
Why it is optimal: undoing the most negative element restores non-negativity with the fewest possible removals, and removing one element never reduces the count below what any other single removal would give. The heap size at the end is the maximum achievable count.

Pattern: Greedy + min-heap ("regret" / exchange argument) **Time:** $O(n \log n)$ **Space:** $O(n)$

```cpp
int maxTransactions(vector<long long>& t) {
    priority_queue<long long, vector<long long>, greater<long long>> pq;
    long long bal = 0;
    for (long long x : t) {
        pq.push(x);
        bal += x;
        if (bal < 0) { // undo the worst pick so far
            bal -= pq.top();
            pq.pop();
        }
    }
    return (int)pq.size();
}
```

Dry run

x	bal after push	bal ¡ 0?	popped	heap size
5	5	no	–	1
−8	−3	yes	−8	1
3	8	no	–	2
−2	6	no	–	3
−7	−1	yes	−7	3

Answer = 3 (namely $5, 3, -2$ with prefixes $5, 8, 6$).

Related

LC 630 Course Schedule III and LC 502 IPO are the same exchange argument. Learn this pattern — it appears in almost every Indian product-company OA.

4 Functional Graph / Permutation Cycle

Problem

Given an array P, the sequence is defined by $Q_1 = P_1$, $Q_k = P[Q_{k-1}]$. Determine the requested value once the sequence enters a cycle, or return -1 if it never resolves.

Key idea. An array where every index has exactly one outgoing edge $i \to P[i]$ is a *functional graph*. Every walk is a "rho" shape: a tail followed by exactly one cycle. So you never need more than n steps — stamp each visited node with the step number at which you saw it. The first repeat gives you both the cycle entry point and the cycle length in one pass.

Pattern: Functional graph traversal / visited stamping **Time:** $O(n)$ **Space:** $O(n)$

```cpp
// Returns {cycleStart, cycleLength} for the walk beginning at `src`,
// or {-1,-1} if the walk leaves the array (invalid index).
pair<int,int> findCycle(const vector<int>& P, int src) {
    int n = P.size();
    vector<int> stamp(n, -1); // step at which node was first seen
    int cur = src, step = 0;
    while (cur >= 0 && cur < n) {
        if (stamp[cur] != -1) // seen before => cycle found
            return {cur, step - stamp[cur]};
        stamp[cur] = step++;
        cur = P[cur];
    }
    return {-1, -1};
}

// Floyd variant: O(1) extra memory, useful if n is huge.
pair<int,int> floydCycle(const vector<int>& P, int src) {
    int slow = P[src], fast = P[P[src]];
    while (slow != fast) { slow = P[slow]; fast = P[P[fast]]; }
    slow = src;
    while (slow != fast) { slow = P[slow]; fast = P[fast]; }
    int start = slow, len = 1;
    for (int v = P[start]; v != start; v = P[v]) ++len;
    return {start, len};
}
```

Interview follow-up. "Now do it in $O(1)$ memory." That is the Floyd version above. Expect it.

5 Minimise Tree Height with K Cut-and-Attach Operations

Problem

A rooted tree is given. In at most K operations you may cut any subtree from its parent and re-attach it directly under the root. Minimise the final height.

Key idea — binary search on the answer + greedy bottom-up cutting. Height is monotone in the number of cuts allowed: if height h is achievable with c cuts, then $h+1$ is achievable with at most c cuts. So binary search $h \in [1, n]$ and ask "can I reach height $\le h$ using $\le K$ cuts?"
For a fixed h, run a post-order DFS returning the height of the *remaining* subtree rooted at v. If that height would reach h at a non-root node, cut v immediately: this is the deepest-first greedy, and because we cut bottom-up, every cut subtree already has height $< h$, so hanging it under the root keeps it within budget. Return -1 from a cut node so the parent does not count it.

Pattern: Binary search on answer + greedy DFS **Time:** $O(n \log n)$ **Space:** $O(n)$

```
int n, K;
vector<vector<int>> adj; // children lists, node 0 = root

int cuts;
// returns height (in edges) of the part of subtree(v) still attached to v
int dfs(int v, int h) {
    int best = 0;
    for (int c : adj[v]) {
        int hc = dfs(c, h);
        if (hc >= 0) best = max(best, hc + 1);
    }
    if (v != 0 && best + 1 >= h) { // attaching v to its parent would exceed h
        ++cuts; // cut v, hang subtree(v) under the root
        return -1;
    }
    return best;
}

bool feasible(int h) { cuts = 0; dfs(0, h); return cuts <= K; }

int minHeight() {
    int lo = 1, hi = n, ans = n;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (feasible(mid)) { ans = mid; hi = mid - 1; }
        else lo = mid + 1;
    }
    return ans;
}
```

Watch out. A recursive DFS on $n = 10^5$ in a chain will stack-overflow on some judges. Convert to an explicit stack (iterative post-order) if the constraints are tight.

6 Tree Stones — Make Every Adjacent Difference Exactly 1

Problem

A tree with n nodes, node v holds $s_v \geq 0$ stones. You may only *add* stones. Minimise the total added so that $|f_u - f_v| = 1$ for every edge (u, v).

Key idea — parity is forced, then one bottom-up pass. If adjacent values differ by exactly 1, their parities differ. A tree is bipartite, so the parity of f_v is completely determined by depth(v) once you fix the root's parity. That leaves only *two* global cases: root even, root odd. Solve both and take the minimum.
For a fixed parity assignment, define m_v = the smallest value f_v can take such that a valid assignment of v's whole subtree exists. Bottom-up:

$$m_v \;=\; \max\Big(\text{lift}(s_v,\, \text{par}(v)),\; \max_{c \in \text{child}(v)} \big(m_c - 1\big)\Big)$$

where $\text{lift}(s, p) = s$ if $s \equiv p \pmod 2$, else $s + 1$. Note m_c has the opposite parity to v, so $m_c - 1$ already has v's parity — the max stays parity-correct. Also, if f_v works then so does $f_v + 2$ (raise the whole subtree by 2), which is exactly what makes the bottom-up minimum sufficient. Then a top-down pass fixes actual values: $f_{\text{root}} = m_{\text{root}}$, and for each child pick the cheaper of $f_v - 1$ and $f_v + 1$ that is still $\geq m_c$.

Pattern: Bipartite parity + two-pass tree DP **Time:** $O(n)$ **Space:** $O(n)$

```
int n;
vector<vector<int>> adj; // undirected
vector<long long> s, m, f;
vector<int> par; // parent

long long lift(long long x, int p) { return ((x & 1LL) == p) ? x : x + 1; }

// pass 1: minimal feasible value, bottom-up
void down(int v, int p, int parity) {
    par[v] = p;
    long long best = lift(s[v], parity);
    for (int c : adj[v]) if (c != p) {
        down(c, v, parity ^ 1);
        best = max(best, m[c] - 1);
    }
    m[v] = best;
}

// pass 2: assign concrete values, top-down
void up(int v, int p) {
    for (int c : adj[v]) if (c != p) {
        f[c] = (f[v] - 1 >= m[c]) ? f[v] - 1 : f[v] + 1;
        up(c, v);
    }
}

long long solveWithRootParity(int parity) {
    m.assign(n, 0); f.assign(n, 0); par.assign(n, -1);
    down(0, -1, parity);
    f[0] = m[0];
    up(0, -1);
    long long add = 0;
    for (int v = 0; v < n; ++v) add += f[v] - s[v];
    return add;
}

```

```
long long minStonesToAdd() {
    return min(solveWithRootParity(0), solveWithRootParity(1));
}
```

Sanity check. Path $1-2-3$ with $s = [1, 1, 1]$. Root parity 1 (odd): $m_3 = 1$, $m_2 = \max(\text{lift}(1, 0), m_3 - 1) = \max(2, 0) = 2$, $m_1 = \max(\text{lift}(1, 1), m_2 - 1) = \max(1, 1) = 1$. So $f = [1, 2, 1]$, total added $= 1$. Root parity 0 gives $f = [2, 1, 2]$, added $= 3$. Minimum is 1. Correct.

7 Fill Zeros So Adjacent Absolute Difference Is At Most 1

Problem

Array `arr` of length n; some entries are 0 (blank). Fill each blank with an integer in $[1, M]$ so that $|arr_i - arr_{i+1}| \le 1$ for all i. Count the valid completed arrays modulo $10^9 + 7$.

Key idea — DP over (position, value), then kill the inner loop with a prefix sum. Let $dp[i][v]$ = number of ways to fill positions $0..i$ with $arr_i = v$. The transition

$$dp[i][v] = dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]$$

is a fixed-width window sum, so precompute $\text{pre}[v] = \sum_{u \le v} dp[i-1][u]$ and read the window as $\text{pre}[v+1] - \text{pre}[v-2]$. Naively $O(nM)$ with a 3-term sum is already fine, but the prefix version generalises immediately if the constraint becomes $|diff| \le k$ — which is the standard follow-up. Fixed positions simply restrict v to the single given value.

Pattern: DP with prefix-sum optimised transition **Time:** $O(nM)$ **Space:** $O(M)$

```cpp
const long long MOD = 1000000007LL;

long long countFillings(vector<int>& arr, int M) {
    int n = arr.size();
    vector<long long> dp(M + 2, 0), nxt(M + 2, 0), pre(M + 3, 0);

    // base: position 0
    for (int v = 1; v <= M; ++v)
        dp[v] = (arr[0] == 0 || arr[0] == v) ? 1 : 0;

    for (int i = 1; i < n; ++i) {
        pre[0] = 0;
        for (int v = 1; v <= M; ++v) pre[v] = (pre[v-1] + dp[v]) % MOD;

        for (int v = 1; v <= M; ++v) {
            if (arr[i] != 0 && arr[i] != v) { nxt[v] = 0; continue; }
            int hi = min(M, v + 1), lo = max(1, v - 1);
            nxt[v] = (pre[hi] - pre[lo-1] % MOD + MOD) % MOD;
        }
        swap(dp, nxt);
    }

    long long ans = 0;
    for (int v = 1; v <= M; ++v) ans = (ans + dp[v]) % MOD;
    return ans;
}
```

Follow-up you should be ready for. If $M \le 10^9$ but n is small, the answer is a polynomial-in-M / matrix-power problem instead: build the tridiagonal transfer matrix and exponentiate in $O(M^3 \log n)$ — only viable for small M. If both are large the problem is unsolvable as stated, and saying so is a correct answer.

8 Killing Enemies Under an Energy Threshold

Problem

Arrays `layers` and `energy` of length n, and an initial energy K. Starting from index i and moving right, you fight enemies in order; after defeating the enemy at index j your remaining energy must be at least `energy[j]`. Return, for every starting index, the maximum number of enemies killed.

Reformulating the condition

Let $\mathrm{pre}[j] = \sum_{k \le j} \texttt{layers}[k]$ with $\mathrm{pre}[-1] = 0$. Starting at i and having just killed j:

$$K - \big(\mathrm{pre}[j] - \mathrm{pre}[i-1]\big) \;\ge\; \texttt{energy}[j] \quad\Longleftrightarrow\quad \underbrace{\mathrm{pre}[j] + \texttt{energy}[j]}_{g[j]} \;\le\; K + \mathrm{pre}[i-1].$$

Key idea. The starting index has been separated from the running index. For a fixed i the threshold $T_i = K + \mathrm{pre}[i-1]$ is a constant, so the answer is the largest j such that $\max_{k \in [i,j]} g[k] \le T_i$. Range max plus binary search — a sparse table gives $O(1)$ queries and $O(n \log n)$ total.

Pattern: Prefix sums + sparse table range max + binary search **Time:** $O(n \log n)$ **Space:** $O(n \log n)$

```cpp
struct SparseMax {
    vector<vector<long long>> t; vector<int> lg;
    SparseMax(const vector<long long>& a) {
        int n = a.size(), LOG = 1; while ((1 << LOG) <= n) ++LOG;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i/2] + 1;
        t.assign(LOG, vector<long long>(n));
        t[0] = a;
        for (int k = 1; k < LOG; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                t[k][i] = max(t[k-1][i], t[k-1][i + (1 << (k-1))]);
    }
    long long query(int l, int r) const { // inclusive
        int k = lg[r - l + 1];
        return max(t[k][l], t[k][r - (1 << k) + 1]);
    }
};

vector<int> maxKills(vector<long long>& layers, vector<long long>& energy, long long K) {
    int n = layers.size();
    vector<long long> pre(n), g(n);
    for (int i = 0; i < n; ++i) {
        pre[i] = (i ? pre[i-1] : 0) + layers[i];
        g[i] = pre[i] + energy[i];
    }
    SparseMax sm(g);

    vector<int> ans(n, 0);
    for (int i = 0; i < n; ++i) {
        long long T = K + (i ? pre[i-1] : 0);
        if (g[i] > T) { ans[i] = 0; continue; } // cannot even kill the first
        int lo = i, hi = n - 1, best = i;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (sm.query(i, mid) <= T) { best = mid; lo = mid + 1; }
            else hi = mid - 1;
        }
        ans[i] = best - i + 1;
    }
```

```
    return ans;
}
```

Verification against the sample

$n = 3,\ K = 10,\ \texttt{layers} = [5, 8, 1],\ \texttt{energy} = [5, 2, 1]$. Then pre $= [5, 13, 14]$ and $g = [10, 15, 15]$.

i	$T_i = K + \text{pre}[i-1]$	largest j with $\max g[i..j] \le T_i$	answer
0	$10 + 0 = 10$	$g[0] = 10 \le 10$, $g[1] = 15 > 10 \Rightarrow j = 0$	1
1	$10 + 5 = 15$	$g[1] = 15$, $g[2] = 15$, both $\le 15 \Rightarrow j = 2$	2
2	$10 + 13 = 23$	$g[2] = 15 \le 23 \Rightarrow j = 2$	1

Output $[1, 2, 1]$, matching the expected result.

9 Warehouse Robot — Minimum Cost to Clear All Packages

Problem

N bays in a line, `packages[i]` units at bay i. The robot picks one base station — bay 0 or bay $N-1$ — and keeps it for all trips. Per trip it travels to a target bay i, picks up at most 1 unit there *and* at most 1 unit from every intermediate bay it passes, then returns. Travel cost of a trip is the 1-indexed distance to the furthest bay visited. Handling cost is 1 per unit. Minimise total cost.

Derivation (this is what earns the marks)

Handling cost is fixed: $\sum_i \texttt{packages}[i]$. Only travel is optimisable.

Take the base at the **right end**, index $N-1$. A trip whose furthest bay is i costs $N-i$, and it removes one unit from *every* bay in $[i, N-1]$ that still has stock. Two observations:

1. Bay i needs at least $\texttt{packages}[i]$ trips that reach index i or further left. A trip reaching index $j \le i$ also serves bay i. So the number of trips whose furthest index is $\le i$ must be at least $\max_{j \le i} \texttt{packages}[j]$ — the **prefix maximum**.
2. Rewrite the cost of a single trip: a trip with furthest index f costs $N-f$, which is exactly the count of indices in $[f, N-1]$. Summing over all trips and swapping the order of summation,

$$\text{TravelCost} \;=\; \sum_{i=0}^{N-1} \#\{\text{trips that reach index } i\} \;=\; \sum_{i=0}^{N-1} \max_{j \le i} \texttt{packages}[j].$$

The prefix-maximum schedule is simultaneously necessary (by 1) and achievable (make exactly $\max_{j \le i} p_j$ trips reach index i, always grabbing every available intermediate unit), so it is optimal. By symmetry, base at the left end gives $\sum_i \max_{j \ge i} p_j$, the **suffix maximum**.

$$\boxed{\text{Answer} \;=\; \sum_i p_i \;+\; \min\Big(\textstyle\sum_i \max_{j \le i} p_j,\;\; \sum_i \max_{j \ge i} p_j\Big)}$$

No simulation, no heap — one linear pass in each direction.

Pattern: Prefix / suffix maxima + exchange argument **Time:** $O(N)$ **Space:** $O(1)$

```cpp
long long minTotalCost(vector<long long>& p) {
    int n = p.size();
    long long handling = 0;
    for (long long x : p) handling += x;

    long long costRightBase = 0, run = 0; // base at index n-1: prefix max
    for (int i = 0; i < n; ++i) { run = max(run, p[i]); costRightBase += run; }

    long long costLeftBase = 0; run = 0; // base at index 0: suffix max
    for (int i = n - 1; i >= 0; --i) { run = max(run, p[i]); costLeftBase += run; }

    return handling + min(costRightBase, costLeftBase);
}
```

Verification against both samples

packages	handling	prefix-max sum	suffix-max sum	total
$[1,2,3]$	6	$1+2+3=6$	$3+3+3=9$	$6+\min(6,9)=\mathbf{12}$
$[7,4,7]$	18	$7+7+7=21$	$7+7+7=21$	$18+21=\mathbf{39}$

Both match the expected outputs.

Overflow. $N \le 10^5$ and $p_i < 10^9$ give a total near 10^{14}. `long long` throughout.

10 Counter Plus/Minus to Reach a Target

Problem

A counter starts at 1 and increases by 1 every iteration. At each step you may add or subtract the counter's current value from a target. Find the minimum number of steps to reach the target.

Key idea — pure number theory, no search. After k steps the reachable values are exactly $S_k = 1+2+\cdots+k = \frac{k(k+1)}{2}$ minus twice any subset sum. Flipping the sign of a chosen element x changes the total by $2x$, so every reachable value has the same parity as S_k, and every value of that parity in $[-S_k, S_k]$ is reachable.
So: find the smallest k with $S_k \geq |target|$ and $S_k \equiv |target| \pmod 2$. Sign symmetry means only $|target|$ matters. In the worst case you increment k at most twice more after the first $S_k \geq |target|$.

Pattern: Parity + triangular numbers **Time:** $O(\sqrt{target})$ **Space:** $O(1)$

```
int minSteps(long long target) {
    target = llabs(target);
    long long k = 0, sum = 0;
    while (sum < target || ((sum - target) & 1LL)) { ++k; sum += k; }
    return (int)k;
}
```

This is verbatim LC 754 Reach a Number. It has shown up as both an MCQ and a coding question in this OA.

11 MCQ Topics Worked Out

11.1 Minimum swaps to bubble-sort an array

The number of swaps bubble sort performs equals the number of **inversions** — pairs (i, j) with $i < j$ and $a_i > a_j$. Bubble sort only ever swaps adjacent out-of-order elements, and each such swap removes exactly one inversion. Count them with a BIT or merge sort.

```cpp
long long countInversions(vector<int> a) { // merge sort
    int n = a.size();
    vector<int> tmp(n);
    function<long long(int,int)> solve = [&](int l, int r) -> long long {
        if (r - l <= 1) return 0;
        int mid = (l + r) / 2;
        long long inv = solve(l, mid) + solve(mid, r);
        int i = l, j = mid, k = l;
        while (i < mid && j < r) {
            if (a[i] <= a[j]) tmp[k++] = a[i++];
            else { inv += mid - i; tmp[k++] = a[j++]; }
        }
        while (i < mid) tmp[k++] = a[i++];
        while (j < r) tmp[k++] = a[j++];
        copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        return inv;
    };
    return solve(0, n);
}
```

11.2 Identifying a sorting algorithm from a partial array

This MCQ style gives the array after k iterations and asks which algorithms it is consistent with. Memorise the invariants:

Algorithm	State after k passes
Insertion sort	first $k+1$ elements sorted *among themselves*; rest untouched
Selection sort	first k elements are the k globally smallest, in final position
Bubble sort	last k elements are the k globally largest, in final position
Merge sort	blocks of size 2^k are each internally sorted
Quicksort	at least one element sits in its final position with a valid partition around it
Heap sort	last k elements sorted and final; the prefix satisfies the heap property

The classic option set — "insertion but not merge", "merge but not insertion", "either", "neither" — is answered by checking both invariants independently and combining.

11.3 Bridges — edges whose removal disconnects the graph

Tarjan's low-link algorithm. Edge (u, v) with v a DFS child of u is a bridge iff $\text{low}[v] > \text{disc}[u]$: nothing in v's subtree can climb back above u without using that edge.

```cpp
int timer_ = 0;
vector<int> disc, low;
vector<vector<pair<int,int>>> g; // (neighbour, edgeId)
vector<pair<int,int>> bridges;

void tarjan(int u, int pe) {
    disc[u] = low[u] = timer_++;
    for (auto [v, id] : g[u]) {
        if (id == pe) continue; // handles parallel edges correctly
        if (disc[v] == -1) {
            tarjan(v, id);
```

```
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) bridges.push_back({u, v});
        } else {
            low[u] = min(low[u], disc[v]);
        }
    }
}
```

MCQ shortcut. An edge is a bridge iff it lies on no cycle. In a tree every edge is a bridge ($n-1$ of them); in a graph with a single cycle of length c, the answer is $E - c$.

11.4 DBMS points that actually get asked

- **ACID:** Atomicity (all or nothing), Consistency (constraints preserved), Isolation (concurrent = some serial order), Durability (survives crash). Multi-correct questions often pair Durability with WAL and Atomicity with undo logging.
- **Isolation levels:** Read Uncommitted → dirty reads; Read Committed → non-repeatable reads; Repeatable Read → phantoms; Serializable → none. MySQL InnoDB defaults to Repeatable Read; PostgreSQL to Read Committed.
- **Normal forms:** 1NF atomic; 2NF no partial dependency on a composite key; 3NF no transitive dependency; BCNF every determinant is a superkey.
- **Indexes:** B+ tree supports range queries; hash index only equality. A composite index on (a, b) serves `WHERE a=?` and `WHERE a=? AND b=?` but not `WHERE b=?` — leftmost-prefix rule.

Part II — 55 Advanced ML MCQs

12 How to Use This Section

These are pitched deliberately hard — at the level where the naive answer is one of the distractors. Attempt each before reading the solution. Topics are weighted toward what a food-delivery marketplace actually asks: gradient boosting, ranking, evaluation under class imbalance, recommendation, and experimentation.

12.1 Statistics, Probability & Estimation

Q1. You fit ordinary least squares and then add a feature that is an exact linear combination of two existing features. What happens to the training R^2 and to the coefficient estimates?

(a) R^2 strictly increases; coefficients remain unique

(b) R^2 is unchanged; coefficients are no longer unique

(c) R^2 decreases; coefficients shrink

(d) R^2 unchanged; coefficients unchanged

(b). The column space of X is unchanged, so the fitted values and hence R^2 are identical. But $X^\top X$ is now singular, so infinitely many coefficient vectors achieve the same fit. This is exact multicollinearity — the predictions survive, the interpretation does not.

Q2. Maximum likelihood estimation of the variance of a Gaussian, $\hat{\sigma}^2_{MLE} = \frac{1}{n}\sum(x_i - \bar{x})^2$, is biased. The bias arises because:

(a) the Gaussian is not in the exponential family

(b) $\bar{x}$ is itself estimated from the same data, consuming one degree of freedom

(c) MLE is always biased for any parameter

(d) the likelihood is non-convex

(b). Residuals about the *sample* mean are systematically smaller than residuals about the true mean, so the estimator under-shoots by a factor $\frac{n-1}{n}$. Note MLE is not always biased — $\hat{\mu}_{MLE} = \bar{x}$ is unbiased — and MLE is only *asymptotically* unbiased in general.

Q3. You run 20 independent A/B tests, all with a true null effect, at $\alpha = 0.05$. The probability that at least one shows a "significant" result is closest to:

(a) 5% (b) 26% (c) 64% (d) 95%

(c). $1 - 0.95^{20} \approx 0.642$. This is the multiple-comparisons problem; the standard fixes are Bonferroni (conservative, controls FWER) or Benjamini–Hochberg (controls FDR, more powerful at scale).

Q4. CUPED (Controlled-experiment Using Pre-Experiment Data) reduces variance in an A/B test by:

(a) increasing the sample size synthetically

(b) subtracting $\theta(X - \bar{X})$ where X is a pre-period covariate, with θ chosen as $\mathrm{Cov}(Y, X)/\mathrm{Var}(X)$

(c) discarding outlier users

(d) switching from a t-test to a permutation test

(b). CUPED is a control-variate method. The adjusted metric has variance $\mathrm{Var}(Y)(1-\rho^2)$, so a pre-period covariate correlated at $\rho = 0.7$ cuts variance by about half — equivalent to doubling the sample. Crucially X must be measured *before* randomisation, or you bias the estimate.

Q5. In a two-sided test, the p-value is the probability that:

(a) the null hypothesis is true

(b) the alternative is false given the data

(c) a test statistic at least as extreme as observed would occur, assuming the null is true

(d) you make a Type II error

(c). The p-value is $P(\text{data} \mid H_0)$, never $P(H_0 \mid \text{data})$. Confusing the two is the prosecutor's fallacy and is the single most common statistics interview trap.

Q6. A marketplace experiment randomises *users*, but drivers are shared between treatment and control. The main threat to validity is:

(a) selection bias (b) SUTVA violation / interference (c) regression to the mean (d) survivorship bias

(b). The Stable Unit Treatment Value Assumption requires one unit's outcome to be unaffected by another's assignment. When treated users consume shared driver supply, control users get worse ETAs — the treatment effect is overstated. Remedies: switchback (time-sliced) randomisation, cluster randomisation by city or region, or a marketplace equilibrium model.

12.2 Bias–Variance, Regularisation & Model Selection

Q7. Which statement about the bias–variance decomposition is *false*?

(a) It decomposes expected squared error into bias^2, variance, and irreducible noise

(b) It applies exactly to 0–1 classification loss in the same additive form

(c) Bagging primarily reduces variance

(d) Boosting primarily reduces bias

(b). The clean additive decomposition is a property of squared loss. For 0–1 loss the analogous decomposition is not additive in general — variance can even *help* when the bias points the wrong way across the decision boundary.

Q8. L1 regularisation produces exactly-zero coefficients while L2 does not because:

(a) L1 is convex and L2 is not

(b) the L1 ball has corners on the axes, so the loss contour typically first touches it at a vertex

(c) L1 has a larger gradient everywhere

(d) L2 is not differentiable at zero

(b). Geometrically, the ℓ_1 constraint region is a cross-polytope whose vertices lie on the coordinate axes. Analytically, the subgradient of $|w|$ is the whole interval $[-1, 1]$ at $w = 0$, so a range of gradients keeps the coefficient pinned at exactly zero; $\frac{d}{dw} w^2 = 0$ at the origin exerts no such force. Both penalties are convex; L2 *is* differentiable at zero, L1 is not.

Q9. Elastic net is preferred over pure lasso when:

(a) $p \ll n$ and features are independent

(b) groups of features are highly correlated and you want them selected together

(c) you need a closed-form solution

(d) the response is categorical

(b). Lasso arbitrarily picks one feature from a correlated group and zeroes the rest, which makes it unstable across resamples. The added L2 term induces a grouping effect: strongly correlated predictors get similar coefficients. Also, lasso can select at most n features when $p > n$; elastic net removes that ceiling.

Q10. Early stopping in gradient descent on a linear model is approximately equivalent to:

(a) L1 regularisation (b) L2 regularisation (c) dropout (d) data augmentation

(b). For a quadratic objective, running t steps at learning rate η shrinks the component along eigenvector i by roughly $(1-\eta\lambda_i)^t$, which matches ridge shrinkage $\frac{\lambda_i}{\lambda_i+\alpha}$ with $\alpha \approx 1/(\eta t)$. Fewer steps $\Leftrightarrow$ stronger penalty.

Q11. You use k-fold CV to tune hyperparameters, then report the best CV score as your estimate of generalisation error. This estimate is:

(a) unbiased (b) optimistically biased (c) pessimistically biased (d) unbiased only if $k = n$

(b). Selecting the maximum over many noisy estimates is itself a form of fitting — the winner's curse. You need *nested* CV: an inner loop for tuning and an outer loop for the honest estimate.

Q12. Increasing k in k-fold cross-validation from 5 to n (LOOCV) generally:

(a) decreases bias, increases variance of the estimate

(b) decreases both bias and variance

(c) increases bias, decreases variance

(d) has no effect on either

(a). Larger training folds mean less pessimistic bias, but the n models are trained on nearly identical data, so their errors are highly correlated and the averaged estimate has high variance. $k = 5$ or 10 is the usual compromise.

12.3 Trees, Ensembles & Gradient Boosting

Q13. In XGBoost, the optimal leaf weight for a leaf with instance set I is $w^\star = -\frac{\sum_{i\in I} g_i}{\sum_{i\in I} h_i+\lambda}$. The h_i terms are:

(a) first derivatives of the loss (b) second derivatives (Hessian diagonal) (c) sample weights (d) feature importances

(b). XGBoost takes a second-order Taylor expansion of the loss around the current prediction. $g_i = \partial_{\hat{y}}\ell$, $h_i = \partial^2_{\hat{y}}\ell$. For squared loss $h_i = 1$ (reducing to plain gradient boosting); for logistic loss $h_i = p_i(1-p_i)$, which is why confidently-predicted points contribute almost nothing to further splits.

Q14. Random forests decorrelate trees relative to bagging by:

(a) using different loss functions per tree

(b) sampling a random subset of features *at every split*

(c) bootstrapping rows only

(d) pruning each tree to a fixed depth

(b). Bootstrapping rows alone still leaves every tree dominated by the same strong predictor. Restricting to $\sqrt{p}$ (classification) or $p/3$ (regression) candidate features per split forces diversity. Averaging B trees with pairwise correlation ρ gives variance $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ — the $\rho\sigma^2$ floor is exactly what feature subsampling attacks.

Q15. LightGBM's leaf-wise (best-first) growth versus XGBoost's level-wise growth:

(a) always produces identical trees

(b) converges faster per tree but overfits more easily on small data

(c) is slower but more accurate

(d) cannot handle categorical features

(b). Leaf-wise splits whichever leaf gives the largest loss reduction, producing deep asymmetric trees that reach lower training loss with fewer leaves — and overfit small datasets. Control it with `num_leaves` and `min_data_in_leaf` rather than `max_depth` alone.

Q16. GOSS (Gradient-based One-Side Sampling) in LightGBM works by:

(a) dropping all small-gradient instances

(b) keeping all large-gradient instances and randomly sampling small-gradient ones, up-weighting the sampled ones

(c) sampling features rather than rows

(d) bundling mutually exclusive features

(b). Small-gradient instances are already well-fit, so they carry little information — but discarding them outright would shift the data distribution. GOSS keeps a fraction a of the largest gradients, samples b from the rest, and multiplies the sampled ones by $\frac{1-a}{b}$ to keep the information gain estimate unbiased. (Option (d) describes EFB, a different LightGBM trick.)

Q17. Default `feature_importances_` from a scikit-learn random forest (mean decrease in impurity) is biased toward:

(a) binary features (b) high-cardinality and continuous features (c) features appearing early in the column order (d) categorical features

(b). A continuous or high-cardinality feature offers many candidate split points, so by chance alone it can find an impurity reduction — even if it is pure noise. Use permutation importance on a held-out set, or SHAP values, when the number actually matters.

Q18. SHAP values are the unique attribution satisfying local accuracy, missingness, and consistency. "Consistency" means:

(a) attributions sum to the prediction

(b) if a model changes so a feature's marginal contribution never decreases, its attribution never decreases

(c) all features get equal credit

(d) attributions are non-negative

(b). Consistency (monotonicity) is precisely the property that split-count and gain-based importances violate — you can make a feature strictly more important to a model and see its gain importance go *down*. Option (a) describes local accuracy.

Q19. Setting a very small learning rate (`eta`) in gradient boosting without increasing `n_estimators`:

(a) overfits badly (b) underfits (c) has no effect (d) makes training slower but the final model identical

(b). Learning rate and number of trees trade off: total capacity is roughly $\eta \times M$. Shrinking η without adding trees leaves the model far from convergence. The standard recipe is a small η (0.01–0.05) with early stopping on a validation set to choose M.

Q20. Why can gradient boosting on decision trees not extrapolate beyond the range of the training targets?

(a) The loss is non-convex

(b) Predictions are sums of leaf constants, each bounded by the observed target range

(c) Trees are pruned

(d) It can extrapolate if depth is large enough

(b). Every leaf outputs a constant fitted from training residuals, so the ensemble is a piecewise-constant function that flattens outside the training support. This matters for demand or price forecasting with trend: detrend first, or use a linear/GAM component.

12.4 Evaluation, Calibration & Imbalance

Q21. A fraud model has 0.1% positives. ROC-AUC is 0.97 but the model is useless in production. The most likely explanation:

(a) ROC-AUC was computed wrongly

(b) ROC-AUC is insensitive to the huge number of true negatives, so precision at the operating threshold can still be terrible

(c) the model is overfitting

(d) the classes should be balanced by discarding negatives

(b). FPR has the (enormous) negative count in its denominator, so even a very high absolute number of false positives barely moves the ROC curve. PR-AUC, or precision at fixed recall, is the honest metric under extreme imbalance. Note the PR baseline equals the positive rate (0.001 here) while the ROC baseline is always 0.5.

Q22. You train with `scale_pos_weight` to fix imbalance. The resulting probabilities are:

(a) well calibrated (b) systematically too high for the positive class (c) systematically too low (d) unchanged in calibration

(b). Re-weighting or oversampling changes the effective prior, inflating predicted positive probabilities. Ranking is preserved, so AUC is fine, but any downstream expected-value calculation breaks. Fix with a prior-correction on the log-odds, or refit with isotonic/Platt calibration on an untouched validation set.

Q23. Platt scaling versus isotonic regression for calibration:

(a) Isotonic is parametric, Platt is not

(b) Platt fits a sigmoid (2 parameters), isotonic fits any monotone step function and needs much more data

(c) Both require the model to be a tree ensemble

(d) Isotonic can produce non-monotone mappings

(b). Platt is low-variance and appropriate for a few hundred calibration points; isotonic is more flexible but overfits on small sets, producing flat plateaus. Always calibrate on data the model never saw.

Q24. Log loss versus Brier score: which is unbounded, and why does that matter?

(a) Brier, because it squares errors

(b) Log loss, because a confident wrong prediction ($p \to 0$ on a positive) gives $-\log p \to \infty$

(c) Neither is bounded

(d) Both are bounded in $[0, 1]$

(b). Log loss punishes confident mistakes without limit, which is why libraries clip predictions to $[\epsilon, 1-\epsilon]$. Brier is bounded in $[0, 1]$ and is far less sensitive to a handful of catastrophic predictions. Both are proper scoring rules.

Q25. Your offline model beats the incumbent on AUC, but the online A/B test shows no lift. The most probable cause among these:

(a) The online traffic is smaller

(b) Offline evaluation used logged data biased by the incumbent's own policy

(c) AUC is a bad metric in all cases

(d) The random seed differed

(b). This is the feedback-loop / off-policy evaluation problem: you only observe outcomes for items the current system chose to show. Counterfactual estimators (IPS, doubly robust) or an exploration slice in production are the standard corrections.

Q26. For a highly imbalanced problem, which threshold-free metric changes when you subsample the negative class?

(a) ROC-AUC (b) PR-AUC (c) both (d) neither

(b). ROC-AUC is invariant to class ratio because TPR and FPR are computed within each class. Precision mixes the classes, so PR-AUC moves as the base rate moves — meaning PR curves from differently-sampled datasets are not comparable.

12.5 Optimisation & Neural Networks

Q27. AdamW differs from Adam + L2 penalty because:

(a) AdamW uses a different momentum term

(b) In Adam, the L2 gradient is divided by the adaptive second-moment term, so effective decay differs per parameter; AdamW decouples decay from the gradient

(c) AdamW does not use bias correction

(d) They are mathematically identical

(b). Adding λw to the gradient means it gets scaled by $1/(\sqrt{\hat{v}} + \epsilon)$ along with everything else, so parameters with large historical gradients receive weaker regularisation — the opposite of the intent. AdamW applies $w \leftarrow w - \eta\lambda w$ as a separate step.

Q28. Batch normalisation at inference time uses:

(a) statistics of the current test batch

(b) running (exponential moving average) statistics accumulated during training

(c) statistics of the full training set recomputed each time

(d) no normalisation at all

(b). Using test-batch statistics would make a prediction depend on which other examples happen to share the batch — broken for batch size 1 and for online serving. This train/eval mismatch is also the reason BatchNorm behaves badly with very small batches, where GroupNorm or LayerNorm is preferred.

Q29. Dropout with rate p at training time requires which adjustment so train and test expectations match?

(a) Multiply activations by p at test time

(b) Divide surviving activations by $(1 - p)$ at training time (inverted dropout)

(c) Nothing — expectations already match

(d) Add p to each activation at test time

(b). Inverted dropout is the standard implementation: scaling happens during training so inference is a plain forward pass with no extra work. Either rescaling works mathematically, but (b) is what every framework does.

Q30. He (Kaiming) initialisation uses variance $2/n_{in}$ rather than Xavier's $1/n_{in}$ because:

(a) ReLU zeroes roughly half the activations, halving the variance passed forward

(b) ReLU networks are deeper

(c) It speeds up matrix multiplication

(d) It prevents dead neurons entirely

(a). Xavier assumes a symmetric activation with unit derivative near zero (tanh). ReLU discards the negative half, so the factor of 2 restores the forward variance. Getting this wrong in a 50-layer network causes activations to vanish geometrically.

Q31. In self-attention, the scores are divided by $\sqrt{d_k}$ because:

(a) it normalises the output to unit norm

(b) the dot product of two independent d_k-dimensional unit-variance vectors has variance d_k, so without scaling softmax saturates and gradients vanish

(c) it makes the operation associative

(d) it reduces the parameter count

(b). Large-magnitude logits push softmax toward a one-hot distribution whose Jacobian is nearly zero. Scaling keeps the variance at $O(1)$ regardless of head dimension.

Q32. The computational complexity of vanilla self-attention in sequence length n and model dimension d is:

(a) $O(nd)$ (b) $O(n^2 d)$ (c) $O(nd^2)$ (d) $O(n \log n \cdot d)$

(b). The $QK^\top$ product is $n \times n$ with d multiply-adds per entry. Memory is $O(n^2)$ unless you use FlashAttention, which recomputes tiles to get $O(n)$ memory with the same FLOP count.

Q33. A KV cache during autoregressive decoding saves computation by:

(a) caching the softmax output

(b) storing past keys and values so each new token attends in $O(nd)$ instead of recomputing all n positions

(c) quantising the weights

(d) skipping layers

(b). Keys and values for previous tokens never change under causal masking. The cost is memory: cache size grows linearly with sequence length and batch, which is what multi-query and grouped-query attention were invented to shrink.

Q34. LoRA fine-tuning updates W as $W + BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$. At initialisation, A is random and B is zero because:

(a) it saves memory

(b) it makes the adapter start as an exact identity so training begins from the pretrained model

(c) zero-initialising both would also work

(d) it enforces orthogonality

(b). $BA = 0$ at step 0, so the initial forward pass is unchanged. Zero-initialising *both* would make the gradient of each factor zero as well — the product would never leave the origin. Exactly one factor must be zero.

Q35. Gradient clipping by global norm addresses:

(a) vanishing gradients (b) exploding gradients (c) dead ReLUs (d) internal covariate shift

(b). Clipping rescales the whole gradient vector when its norm exceeds a threshold, preserving direction while bounding step size. It does nothing for vanishing gradients — those need residual connections, better initialisation, or gating.

Q36. You double the batch size. To keep the SGD noise scale roughly constant, the learning rate should be:

(a) halved (b) doubled (linear scaling) (c) multiplied by $\sqrt{2}$ (d) unchanged

(b). The linear scaling rule, with a warmup period to avoid instability early in training. The $\sqrt{2}$ rule is the alternative prescription that follows from matching gradient-noise variance under some assumptions; linear scaling is what large-batch training in practice uses, and interviewers expect it.

12.6 Recommendation, Ranking & Marketplace ML

Q37. In a two-tower retrieval model, the user tower and item tower cannot attend to each other. This restriction exists because:

(a) it improves accuracy

(b) item embeddings must be precomputable and indexable in an ANN structure for sub-linear retrieval

(c) cross-attention is not differentiable

(d) it reduces the parameter count

(b). Retrieval must score millions of items in milliseconds. Independence lets you precompute all item vectors offline and serve with HNSW or IVF-PQ. Cross-features are then reintroduced in the *ranking* stage, which only sees a few hundred candidates.

Q38. In-batch negative sampling for retrieval introduces a bias that is usually corrected by:

(a) increasing the batch size only

(b) subtracting log(sampling probability) from the logits (logQ correction)

(c) L2 normalising the embeddings

(d) using a larger learning rate

(b). Popular items appear as in-batch negatives far more often than their true frequency warrants, so the model over-penalises them. The logQ (sampled softmax) correction removes the popularity bias analytically.

Q39. NDCG@k is preferred over precision@k for ranking because:

(a) it is easier to compute

(b) it uses graded relevance and applies a position discount, so ordering *within* the top-k matters

(c) it is threshold-free while precision is not

(d) it does not require ground truth

(b). Precision@k is invariant to permutations inside the top-k; NDCG's $1/\log_2(i+1)$ discount rewards putting the best result first. Normalisation by the ideal DCG makes queries with different numbers of relevant items comparable.

Q40. Pointwise, pairwise, and listwise learning-to-rank. LambdaMART is:

(a) pointwise regression on relevance labels

(b) pairwise, with gradients reweighted by the NDCG change from swapping the pair

(c) purely listwise with a differentiable NDCG

(d) a neural ranker

(b). LambdaMART is gradient-boosted trees over "lambda" gradients: the RankNet pairwise gradient multiplied by $|\Delta\text{NDCG}|$ for that swap. This sidesteps the fact that NDCG is a step function with zero gradient almost everywhere. It remains a very strong baseline on tabular ranking features.

Q41. Position bias in click logs means:

(a) users click randomly

(b) items shown higher get more clicks regardless of relevance, so naive click-through training reinforces the existing ranking

(c) clicks are missing at random

(d) the CTR is always overestimated

(b). The fix is inverse propensity scoring: weight each click by $1/\hat{p}(\text{examination} \mid \text{position})$, where the propensity is estimated by result randomisation or an intervention-harvesting scheme.

Q42. The cold-start problem for a brand-new restaurant is best mitigated by:

(a) matrix factorisation on the interaction matrix

(b) content-based features (cuisine, price band, geo, menu text embeddings) in a hybrid model

(c) increasing the embedding dimension

(d) dropping the restaurant until it has data

(b). Pure collaborative filtering has no signal for an item with zero interactions — its latent vector is untrained. Content features generalise across items, and a bandit-style exploration budget accelerates learning of the true CTR.

Q43. Maximal Marginal Relevance (MMR) trades off relevance and diversity as $\arg\max_i [\lambda \cdot rel(i) - (1-\lambda)\max_{j \in S} sim(i,j)]$. Setting $\lambda = 1$ gives:

(a) maximum diversity (b) pure relevance ranking with no diversity control (c) random ordering (d) undefined behaviour

(b). The diversity penalty vanishes, so MMR degenerates to sorting by relevance — which on a food app returns ten near-identical biryani listings. Diversity is a business metric, not a modelling nicety.

Q44. For predicting delivery ETA, optimising mean absolute error rather than mean squared error is often chosen because:

(a) MAE is differentiable everywhere

(b) MAE targets the conditional median and is far less influenced by rare extreme delays

(c) MAE always gives lower error values

(d) MSE cannot be used with gradient boosting

(b). MSE targets the conditional mean and lets a few 90-minute outliers dominate. MAE is *not* differentiable at zero (option (a) is false). Even better in practice: quantile/pinball loss, so you can promise a P80 ETA and control the under-promise rate explicitly.

Q45. Predicting the 80th-percentile ETA requires which loss?

(a) MSE (b) MAE (c) pinball loss with $\tau = 0.8$ (d) Huber loss

(c). Pinball loss $L_\tau(y, \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y}))$ is minimised at the τ-quantile. It penalises under-prediction $4\times$ more than over-prediction at $\tau = 0.8$, which is exactly the asymmetry a delivery promise needs.

12.7 Features, Leakage & Production

Q46. Which of these is a data leak?

(a) Standardising features using mean and standard deviation computed on the full dataset before splitting

(b) One-hot encoding categorical features

(c) Using a validation set for early stopping

(d) Log-transforming a skewed feature

(a). Test-set statistics have entered the training pipeline. Fit every transformer on the training fold only and apply it to validation and test — which is precisely why scikit-learn `Pipeline` objects exist.

Q47. Target (mean) encoding of a high-cardinality categorical feature overfits unless you:

(a) increase model depth

(b) compute the encoding out-of-fold and smooth toward the global mean

(c) one-hot encode instead

(d) drop rare categories

(b). A category with one row gets encoded as that row's own target — perfect leakage. Use K-fold out-of-fold encoding plus shrinkage $\frac{n\bar{y}_c + m\bar{y}}{n+m}$. CatBoost's ordered target statistics automate this.

Q48. Training-serving skew most commonly arises from:

(a) different random seeds

(b) feature computation logic implemented twice (batch pipeline vs. online service) and drifting apart

(c) different GPU models

(d) using a different optimiser

(b). Same feature name, two implementations, subtly different semantics — a time window computed inclusively offline and exclusively online, say. A feature store with a single shared definition, plus logging of the actual served feature vectors, is the structural fix.

Q49. Your model's input feature distribution shifts but $P(y \mid x)$ stays the same. This is:

(a) concept drift (b) covariate shift (c) label shift (d) no drift at all

(b). Covariate shift changes $P(x)$ only; importance weighting by $P_{\text{new}}(x)/P_{\text{old}}(x)$ can correct it. Concept drift is a change in $P(y \mid x)$ and generally requires retraining — no reweighting saves you.

Q50. Shadow deployment means:

(a) deploying to a fraction of users

(b) running the new model on live traffic and logging its outputs without serving them

(c) training on production data

(d) deploying to a staging environment with synthetic traffic

(b). Shadow mode validates latency, throughput, and prediction distribution against real traffic at zero user risk. Option (a) describes a canary release — interviewers often ask you to distinguish the two.

12.8 Miscellaneous Hard Ones

Q51. PCA applied to features on wildly different scales without standardisation will:

(a) work fine, since PCA is scale invariant

(b) be dominated by the highest-variance (largest-unit) feature

(c) fail to converge

(d) produce complex eigenvalues

(b). PCA maximises variance in the raw units, so a feature measured in rupees dwarfs one measured in fractions. Standardising is equivalent to running PCA on the correlation matrix instead of the covariance matrix. Eigenvalues of a real symmetric covariance matrix are always real, so (d) is impossible.

Q52. k-means is guaranteed to converge but not to the global optimum because:

(a) the objective is non-convex and the algorithm is coordinate descent, so it lands in a local minimum

(b) the distance metric is wrong

(c) it uses random numbers

(d) k is unknown

(a). Each assignment and update step weakly decreases within-cluster sum of squares, and there are finitely many partitions, so it terminates. But the objective is NP-hard globally. Use k-means++ seeding and multiple restarts.

Q53. The curse of dimensionality means that in high dimensions:

(a) all algorithms become slower

(b) the ratio of nearest to farthest neighbour distance approaches 1, making distance-based methods lose discriminative power

(c) data becomes linearly separable

(d) variance decreases

(b). Concentration of measure: pairwise distances converge to a common value, so kNN and k-means degrade. Interestingly, (c) is *also* true in a narrow sense — n points in general position in $d \geq n - 1$ dimensions are always linearly separable, which is why high-dimensional models overfit so readily.

Q54. Naive Bayes often performs surprisingly well on text despite its independence assumption being obviously false because:

(a) words really are independent

(b) classification only needs the correct *argmax*, and the probability estimates can be badly wrong while still ranking classes correctly

(c) text is low dimensional

(d) it uses a kernel

(b). Dependence between correlated words inflates the log-probabilities of both classes in a similar direction, often leaving the decision unchanged. The estimates themselves are wildly overconfident — so never use raw Naive Bayes probabilities for anything requiring calibration.

Q55. You have 500 labelled and 500,000 unlabelled examples. Which approach is most likely to help?

(a) Train a deep network from scratch on the 500 labels

(b) Self-supervised pretraining or a pretrained embedding model, then fine-tune a light head on the 500 labels; optionally add pseudo-labelling with confidence thresholds

(c) Discard the unlabelled data

(d) Oversample the labelled data 1000×

(b). Representation learning is what the unlabelled pool is for. Option (d) adds zero information and only overfits faster. If you do pseudo-label, hold out a clean validation set that never receives pseudo-labels, or you will not notice confirmation bias compounding.

13 Final Checklist

Two days before the OA

- Re-solve Q2 (regret greedy) and Q8 (prefix/suffix maxima) from memory — these two patterns generalise the furthest.
- Sparse table, Tarjan bridges, and inversion counting: have templates memorised, not looked up.
- Revise the sorting-invariant table in §10.2. Free marks.
- ACID, isolation levels, leftmost-prefix index rule.
- For the ML round: bias–variance, XGBoost second-order objective, ROC vs PR under imbalance, and one crisp story about a recommendation or ETA system you have built.

During the OA

- MCQs first, 15 minutes, no negative marking so never leave a blank.
- `long long` by default on every accumulator.
- Submit a brute force before optimising — partial test cases carry marks.

All solutions and MCQs in this document were written from first principles. Problem statements are paraphrased summaries of publicly shared candidate recollections.